

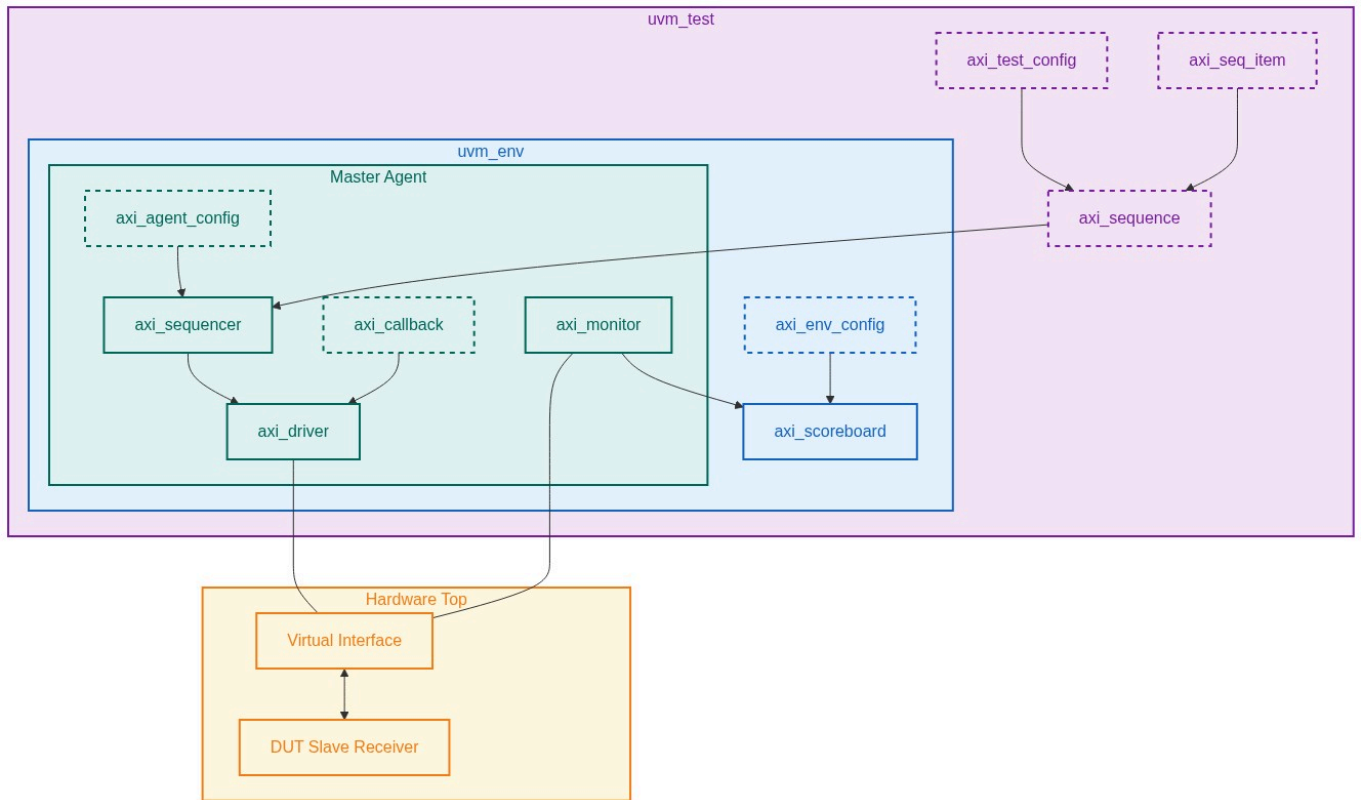
Verification Plan Report

Project: AXI-Stream Master VIP Verification Plan

Version: 17.0

Verification Role: Master

1. UVM Environment Architecture



2. Protocol Configurations

Feature	Configuration Name	Impact
AXI5 Wakeup	Wakeup_Signal_Support	Functional impact requires tracking TWAKEUP states in the driver before TVALID assertion. Performance impact involves verifying exit latency from low power.
Data Width Adaptation	TDATA_Width_Config	Functional impact requires reconfiguring scoreboard comparison to handle upsizing/downsizing null byte insertion.
Interface Protection	Parity_Check_Enable	Functional impact triggers error injection sequences for TDATACHK parity faults. Requires hardened monitor checks.

3. Test Requirements

Verification Plan Report

Index	Feature/Sub	Name	Description
REQ_MST_0 1	Handshake (TVALID Stability)	TVALID asserted until TREADY	The protocol requires that once TVALID is asserted, it must remain HIGH until a valid handshake (TVALID & TREADY) occurs. Pre-mature de-assertion causes data loss. Strategy: Verification involves driving TVALID and randomly stalling TREADY for 1-50 cycles while monitoring TTDATA stability.
REQ_MST_0 2	Packet Boundaries (TLAST Marking)	TLAST End of Packet	TLAST marks the final transfer of a packet. Downstream components use this to trigger frame processing or buffer flushing. Strategy: Generate varied packet lengths (1 to 4096 beats) and verify that TLAST is exclusively asserted on the final beat.
REQ_MST_0 3	Data Integrity (Parity Check)	AXI5 TDATACHK Protection	AXI5-Stream introduces TDATACHK lines for parity. Errors must be detected by the receiver and potentially reported via system interrupts. Strategy: Negative scenario involves flipping a random data bit or parity bit and verifying the Monitor flags a protocol violation.
REQ_MST_0 4	Low Power (Wakeup Handshake)	TWAKEUP Assertion Priority	TWAKEUP must indicate activity and ideally trigger at least one cycle before TVALID. Strategy: Test sequence activates TWAKEUP, waits for programmable latency, then starts TVALID. Monitor checks for TWAKEUP=0 during transfers.

4. Test Cases & Mapping

Index	Scenario	TR Mapping
TC_MST_001	Drive back-to-back 1-beat packets with 0-cycle TREADY delay.	REQ_MST_01, REQ_MST_02
TC_MST_002	Inject 20-cycle TREADY stall and verify TDATA remains constant during VALID phase.	REQ_MST_01
TC_MST_003	Generate Sparse Stream where TKEEP alternates between 0 and 1 every beat.	REQ_MST_02
TC_MST_004	Negative: Force TVALID LOW while TREADY is LOW after initial assertion.	REQ_MST_01
TC_MST_005	Inject single-bit parity fault on TDATACHK during a 16-beat packet.	REQ_MST_03
TC_MST_006	Simultaneous TVALID and TWAKEUP assertion; verify TREADY waits for stabilization.	REQ_MST_04

5. Functional Coverage Groups

Group Name	Description
cg_handshake_stalls	Cross-coverage of TREADY stall durations and TVALID assertion states.
cg_packet_lengths	Binned coverage of packet sizes from 1 beat to max frame depth.

Verification Plan Report

6. Interface Signals

Signal Name	Direction	Width	Description
ACLK	Input	1	Global clock signal.
ARESETn	Input	1	Global active-LOW reset.
TVALID	Output	1	Indicates Transmitter driving valid transfer.
TREADY	Input	1	Indicates Receiver can accept transfer.
TDATA	Output	`TDATA_WIDT H	Primary payload data channel.
TKEEP	Output	`TDATA_WIDT H/8	Byte qualifier for data stream membership.
TLAST	Output	1	Packet boundary marker.
TWAKEUP	Output	1	AXI5 activity indicator.

7. Verification Environment Structure

```
axi_stream_master_vip_tb/  
|-- master_agent/  
|   |-- axi_stream_master_driver.sv  
|   |-- axi_stream_master_monitor.sv  
|   |-- axi_stream_master_agent_config.sv  
|   |-- axi_stream_master_callback.sv  
|   |-- axi_stream_master_sequencer.sv  
|-- env/  
|   |-- axi_stream_master_scoreboard.sv  
|   |-- axi_stream_master_env_config.sv  
|-- sequences/  
|   |-- axi_stream_master_base_sequence.sv  
|   |-- axi_stream_master_seq_item.sv  
|-- top/  
|   |-- axi_stream_master_tb_top.sv  
|   |-- axi_stream_master_test.sv  
|   |-- axi_stream_master_defines.sv
```

8. SystemVerilog Implementation

axi_stream_master_defines.sv (Architecture Blueprint)

COMPONENT: Global Defines

PARAMETERS:

- TDATA_WIDTH: 32 (Configurable to 64/128)
- TID_WIDTH: 8
- HAS_AXI5: 1

Verification Plan Report

- MAX_STALL: 100

LOGIC:

- Centralized parameterization for all Width-dependent signal arrays across the environment components.

axi_stream_master_seq_item.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_seq_item

EXTENDS: uvm_sequence_item

MEMBERS:

- rand logic [`TDATA_WIDTH-1:0] data
- rand logic [`TDATA_WIDTH/8-1:0] keep
- rand bit last
- rand int delay

LOGIC:

- constraint c_delay { delay inside {[0:`MAX_STALL]}; }
- constraint c_keep { keep != 0; } (Baseline validity)

axi_stream_master_driver.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_driver

EXTENDS: uvm_driver #(axi_stream_master_seq_item)

CONFIG_REF: axi_stream_master_agent_config (Source: ConfigDB)

MEMBERS:

- virtual interface axi_stream_if vif
- axi_stream_master_agent_config m_cfg

LOGIC:

- build_phase: Retrieve m_cfg from DB; assign vif handle from m_cfg.vif.
- run_phase:
 - Wait for ARESETn to release.
 - forever:
 - get_next_item(req).
 - Drive TDATA, TKEEP, TLAST.
 - Assert TVALID.
 - Wait for @(posedge vif.ACLK) until (vif.TREADY === 1).
 - item_done(req).

axi_stream_master_agent.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_agent

EXTENDS: uvm_agent

CONFIG_REF: axi_stream_master_agent_config (Source: ConfigDB)

MEMBERS:

- axi_stream_master_driver m_driver
- axi_stream_master_monitor m_monitor
- axi_stream_master_sequencer m_sqr
- axi_stream_master_agent_config m_cfg

LOGIC:

- build_phase:
 - Create m_monitor (unconditional).

Verification Plan Report

```
- if(m_cfg.is_active == UVM_ACTIVE): Create m_driver and m_sqr.  
- connect_phase:  
  - if(m_cfg.is_active == UVM_ACTIVE): m_driver.seq_item_port.connect(m_sqr.seq_item_export).
```

axi_stream_master_test.sv (Architecture Blueprint)

COMPONENT: axi_stream_master_test

EXTENDS: uvm_test

MEMBERS:

```
- axi_stream_master_env m_env  
- axi_stream_master_env_config m_env_cfg  
- axi_stream_master_agent_config m_agent_cfg
```

LOGIC:

```
- build_phase:  
  - Instantiate m_env_cfg and m_agent_cfg.  
  - Set VIF handle and activity flags.  
  - uvm_config_db#(env_cfg)::set(this, "", "env_cfg", m_env_cfg).  
  - uvm_config_db#(agent_cfg)::set(this, "*agent*", "agent_cfg", m_agent_cfg).  
  - Create m_env.
```